Amortized Post-Quantum Secure Aggregation for Federated Learning

Author Name
Affiliation
email

***Abstract*—**

***Index Terms*—post-quantum cryptography, federated learning, ML-KEM, ML-DSA, secure aggregation**

I. Introduction

II. Background

A. Post-Quantum KEMs and Signatures

B. Federated Learning and Update Confidentiality

III. Threat Model

IV. Protocol Design

A. Session Setup

B. Per-Round Update Encryption

C. Re-Keying Schedule

D. Operating Modes

V. Implementation

VI. Experimental Setup

VII. Results

A. Per-Round and Cumulative Crypto Overhead

B. Communication Overhead

C. Effect of Re-Key Interval

D. Scaling with the Number of Clients

E. Model Quality

VIII. Security Evaluation

IX. Discussion and Limitations

X. Related Work

XI. Conclusion

Appendix A
Implementation Notes

- **KEM input lengths unchecked by liboqs-python.** `encap_secret()` accepted a 1,183-byte ML-KEM-768 public key (expected 1,184) without error. *Fix:* wrapper validates public-key and ciphertext lengths; malformed keys are rejected as authentication failures.
- **MITM on the client-only-signed handshake.** An unauthenticated KEM ciphertext let an attacker re-encapsulate and learn the client's session key. *Fix:* server signs its response, bound to $H(init)$, in every mode.
- **Header epoch not covered by the AEAD.** With server-side AAD, a changed header epoch still authenticates; mutation M5 showed the explicit header check is the only defense. *Fix:* check retained and tested.
- **Cold-start timing bias.** First X25519 keygen ≈3.8 ms vs. ≈31 µs warm (lazy OpenSSL init) inflated classical round 0 (8.8 vs. 4.0 ms). *Fix:* untimed warm-up of all primitives before round 0.
- **Participation-dependent epochs.** Counting handshakes would desynchronize clients that skip rounds. *Fix:* scheduled epoch $e = \lfloor r/N \rfloor$; stale or non-increasing handshake epochs rejected.
- **Cadence-confounded baseline.** Classical originally keyed once. *Fix:* classical supports the same N, isolating the primitives.
- **Training cost.** A full MNIST pass took ≈13 s per round (about 9 h for the grid). *Fix:* per-client sample cap (2,000) and a run cache shared across experiments (80 distinct runs instead of 105).
- **Library/binding version coupling.** liboqs and liboqs-python must share major.minor; a missing library triggers a source build. *Fix:* liboqs 0.16.0 (Homebrew) with liboqs-python 0.16.0.1 pinned.
- **Interpreter choice.** System default Python 3.14. *Fix:* virtualenv on Python 3.11 for torch/torchvision wheel availability.
- **Manifest recorded CLI flags, not effective settings.** *Fix:* effective seeds, rounds, K, N, sweeps and sample cap stored and consumed by the asset builder.
- **Output naming.** Naive PQC CSVs were labelled N=none. *Fix:* names use the effective interval.
- **Benchmark conditions.** Laptop on battery at run time. *Fix:* full run deferred to AC power; runs executed sequentially.
- **No TeX toolchain.** *Fix:* `tectonic` installed for diagrams and the skeleton.

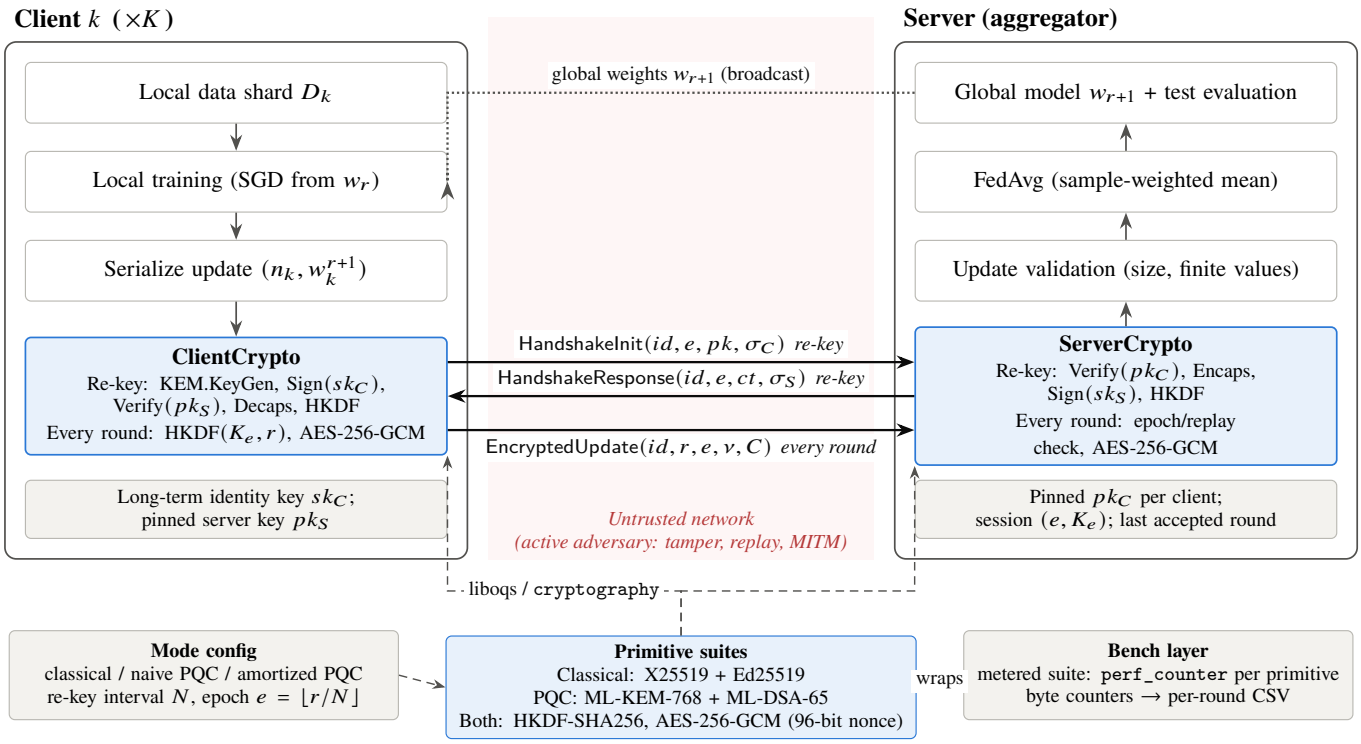


Fig. 1. System architecture.

TABLE I
MAIN COMPARISON OF THE THREE MODES

Mode	N	HS	Crypto (ms)	Handshake (ms)	AES-GCM (ms)	HS KB/client	% round	Acc. (%)
Classical	5	20	38.00 ± 0.61	9.10 ± 0.08	28.89 ± 0.55	0.8	0.089 ± 0.001	98.21 ± 0.06
Naive PQC	1	100	79.87 ± 0.54	50.66 ± 0.84	29.21 ± 0.43	173.6	0.182 ± 0.003	98.21 ± 0.06
Amortized PQC	5	20	39.93 ± 0.60	10.25 ± 0.17	29.68 ± 0.46	34.7	0.091 ± 0.001	98.21 ± 0.06

K = 5, 20 rounds, 5 seeds; times summed over all clients and the server; HS = handshakes; % round = crypto share of wall-clock round time incl. local training, excl. test-set evaluation.

TABLE II
TIME PER PRIMITIVE OVER A SESSION (MS)

Mode	KeyGen	Encaps	Decaps	Sign	Verify
Classical	0.978 ± 0.019	1.126 ± 0.008	0.989 ± 0.005	2.086 ± 0.007	2.924 ± 0.007
Naive PQC	5.231 ± 0.103	2.167 ± 0.023	2.225 ± 0.013	23.938 ± 0.927	10.634 ± 0.045
Amortized PQC	1.140 ± 0.025	0.455 ± 0.008	0.449 ± 0.002	4.749 ± 0.133	2.147 ± 0.010

KEM keygen only; long-term identity keys are generated once at setup.

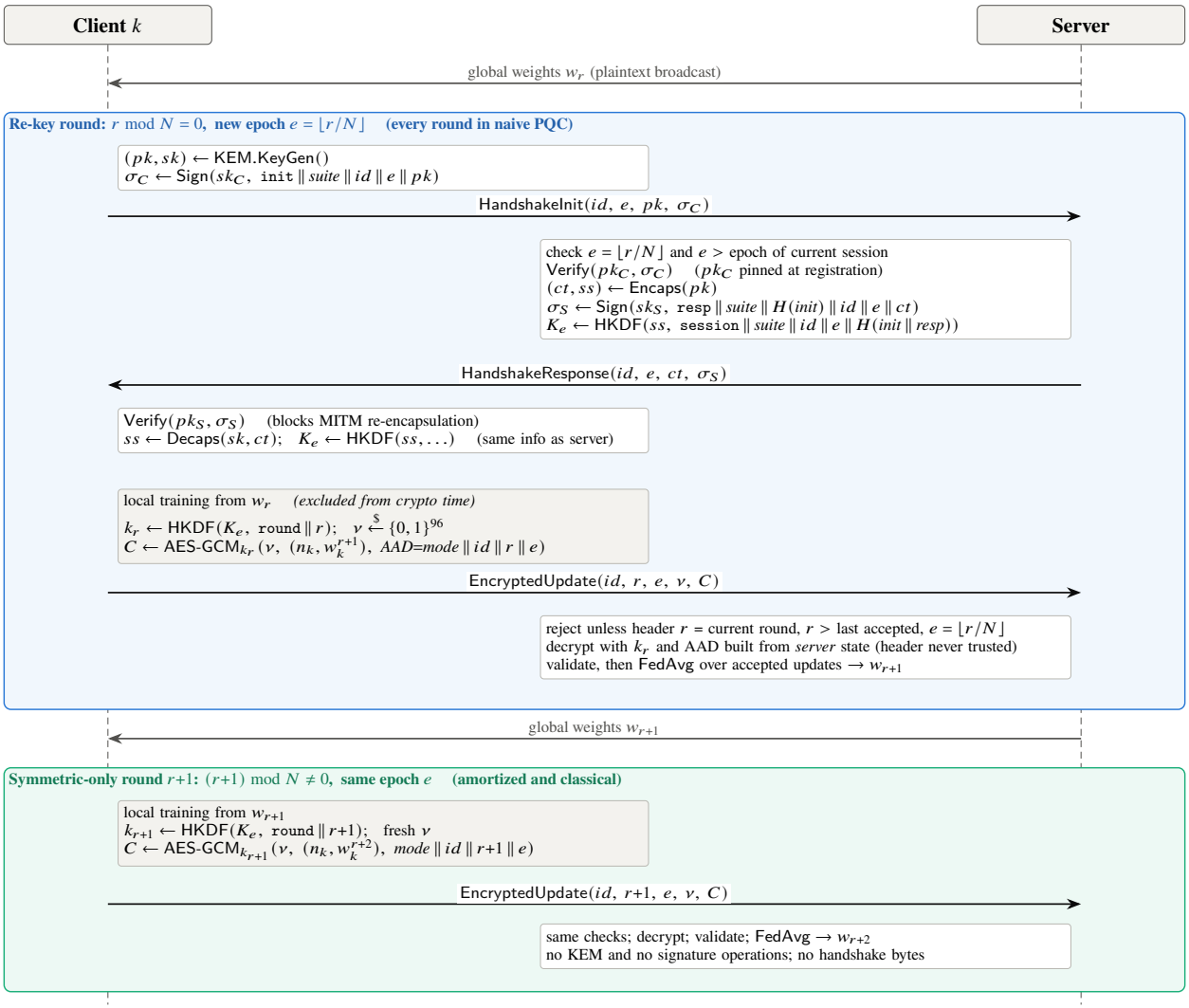


Fig. 2. Message sequence for a re-key round followed by a symmetric-only round.

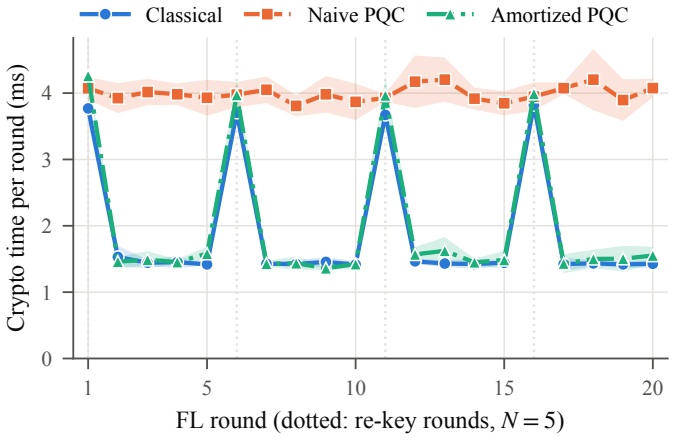


Fig. 3. Crypto time per round by mode (mean ± std over seeds).

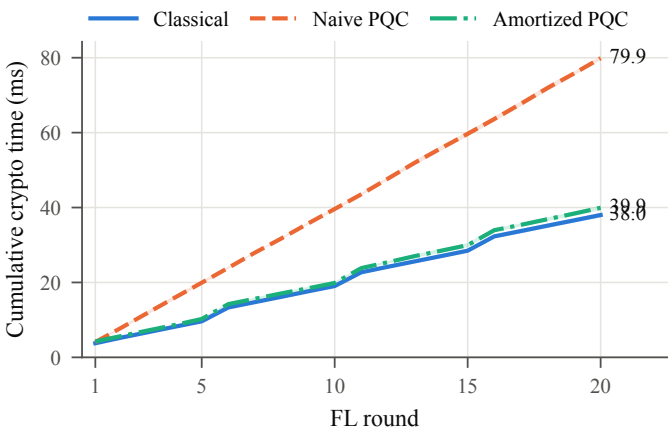


Fig. 4. Cumulative crypto time over rounds by mode.

TABLE III
COMPUTE VERSUS BANDWIDTH OVERHEAD PER ROUND

Mode	Crypto/round (ms)	Crypto % of round	Wire KB/client/round	Overhead B/client/round	Overhead % of payload
Classical	1.900 ± 0.031	0.089 ± 0.001	841.4	123	0.014
Naive PQC	3.994 ± 0.027	0.182 ± 0.003	850.1	9,026	1.048
Amortized PQC	1.997 ± 0.030	0.091 ± 0.001	843.1	1,863	0.216

$K = 5$, $N = 5$ (Naive PQC: $N = 1$), 20 rounds, 5 seeds. Round time is wall-clock including local training, excluding test-set evaluation. Overhead bytes = all protocol bytes beyond the serialized update (KEM keys and ciphertexts, signatures, nonce, GCM tag, framing).

TABLE IV
PER-ROUND PROTOCOL OVERHEAD (% OF MODEL UPDATE) VERSUS UPDATE SIZE

Update size S	Classical, $N = 5$	Naive PQC ($N = 1$)	Amortized PQC, $N = 5$	Amortized PQC, $N = 20$
10 KB	1.2	88.1	18.2	5.08
100 KB	0.12	8.81	1.82	0.508
841 KB (our CNN)	0.0143	1.05	0.216	0.0603
1 MB	0.0117	0.861	0.178	0.0496
10 MB	0.00117	0.0861	0.0178	0.00496

Overhead = $100\,(o + H/N)/S$ % per client per round, with measured $o = 72$ B per update (nonce, GCM tag, framing), $H = 256$ B per classical handshake and $H = 8{,}954$ B per PQC handshake (KEM key and ciphertext, two signatures, framing). Steady state (R a multiple of N).

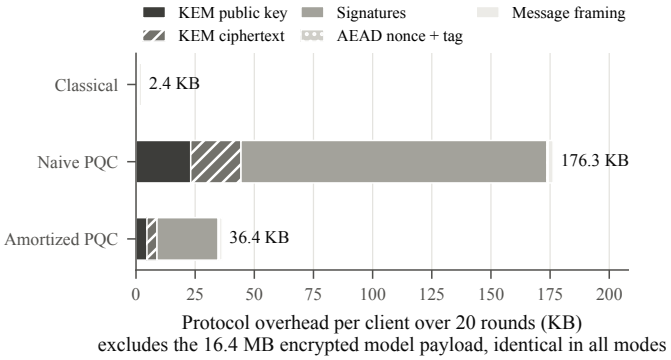


Fig. 5. Protocol bytes on the wire per client, by component.

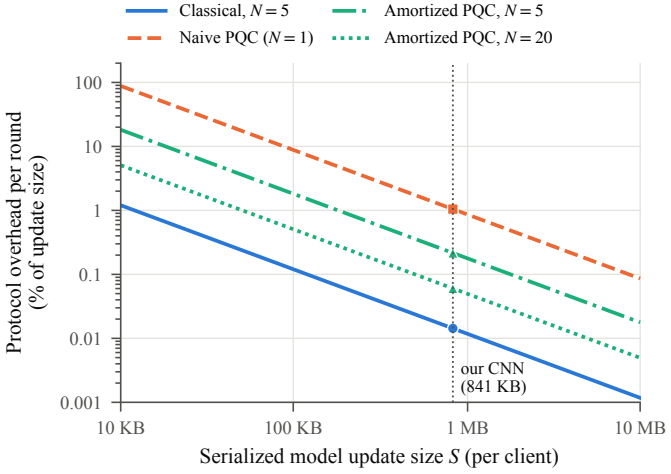


Fig. 6. Per-round protocol overhead versus model update size S, computed as $100\,(o + H/N)/S$ % from measured byte counts: o bytes per update (nonce, GCM tag, framing) and H bytes per handshake per client. Markers: our CNN.

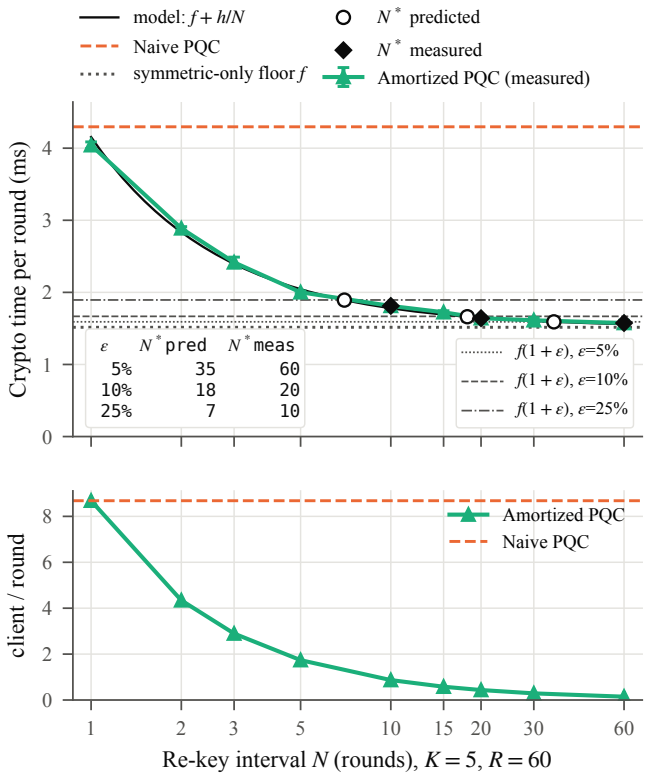


Fig. 7. Overhead versus re-key interval N (amortized PQC) with the cost model $f + h/N$. $N^*(\varepsilon)$: smallest N whose handshake overhead per round is at most εf, predicted as $\lceil h/(\varepsilon f) \rceil$ and measured, for $\varepsilon \in \{5, 10, 25\}$%.

TABLE V
AMORTIZED PQC VERSUS RE-KEY INTERVAL N (TIMES IN MS)

N	HS rounds	Crypto/round	HS overhead/round	Model h/N	% of f
1	60	4.041 ± 0.046	2.563 ± 0.039	2.638	169.1
2	30	2.888 ± 0.024	1.358 ± 0.012	1.319	89.6
3	20	2.416 ± 0.073	0.892 ± 0.029	0.879	58.9
5	12	2.001 ± 0.020	0.529 ± 0.010	0.528	34.9
10	6	1.812 ± 0.027	0.270 ± 0.008	0.264	17.8
15	4	1.724 ± 0.021	0.181 ± 0.005	0.176	12.0
20	3	1.641 ± 0.032	0.133 ± 0.007	0.132	8.8
30	2	1.613 ± 0.037	0.093 ± 0.005	0.088	6.1
60	1	1.574 ± 0.038	0.048 ± 0.005	0.044	3.2
Naive PQC	60	4.296	2.739	–	180.7

ε	N^* predicted $\lceil h/(\varepsilon f)\rceil$	next swept N	N^* measured
5%	35	60	60
10%	18	20	20
25%	7	10	10

$K = 5$, $R = 60$, 5 seeds. $f = 1.516$ ms/round (symmetric-only floor), $h = 2.638 \pm 0.207$ ms (extra cost of one handshake round, 690 rounds pooled). Measured N^*: smallest swept N with handshake overhead per round $\leq \varepsilon f$; – = not reached.

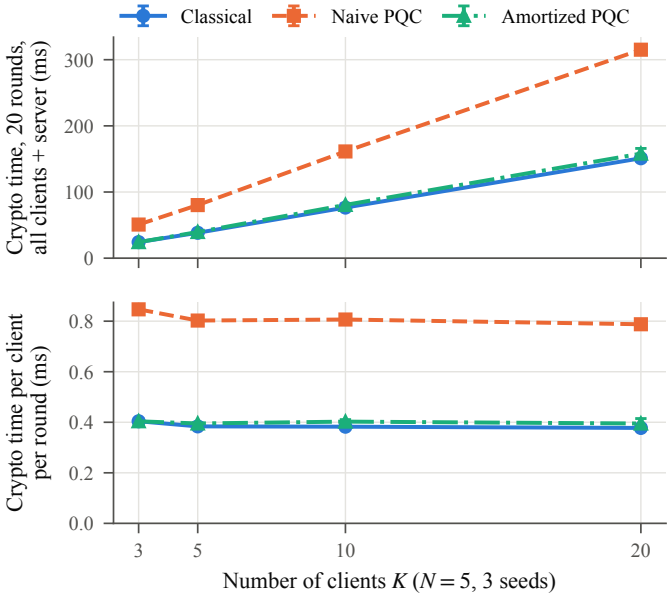


Fig. 8. Crypto overhead versus number of clients K.

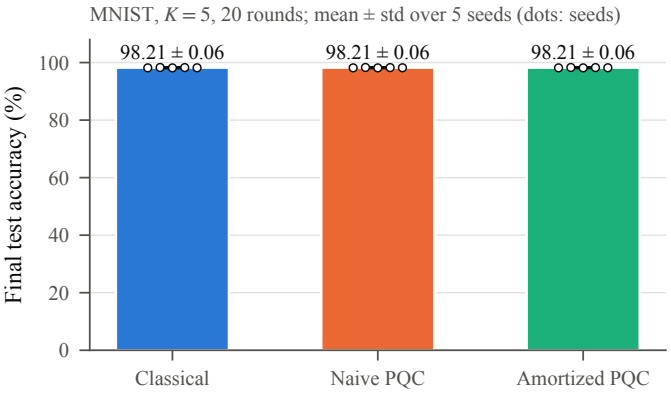


Fig. 9. Final test accuracy per mode (mean ± std over seeds).

TABLE VI
CRYPTO COST VERSUS NUMBER OF CLIENTS K

K	Classical		Naive PQC		Amortized PQC	
	Crypto (ms)	% round	Crypto (ms)	% round	Crypto (ms)	% round
3	24.24 ± 0.59	0.091 ± 0.002	50.81 ± 0.21	0.190 ± 0.001	24.23 ± 0.37	0.091 ± 0.001
5	38.37 ± 0.45	0.089 ± 0.001	80.25 ± 0.22	0.182 ± 0.003	39.53 ± 0.31	0.090 ± 0.001
10	76.50 ± 1.36	0.089 ± 0.002	161.31 ± 1.44	0.188 ± 0.007	80.51 ± 1.60	0.092 ± 0.001
20	151.02 ± 3.99	0.089 ± 0.002	315.11 ± 4.16	0.182 ± 0.001	158.02 ± 7.87	0.091 ± 0.003

$N = 5$ (Naive PQC: $N = 1$), 20 rounds, 3 seeds; times summed over all clients and the server.

TABLE VII
FINAL TEST ACCURACY (%) PER MODE AND SEED

Seed	Classical	Naive PQC	Amortized PQC	max $\lvert\Delta\rvert$ any round
0	98.16	98.16	98.16	0.00
1	98.29	98.29	98.29	0.00
2	98.15	98.15	98.15	0.00
3	98.25	98.25	98.25	0.00
4	98.19	98.19	98.19	0.00
Mean	98.21 ± 0.06	98.21 ± 0.06	98.21 ± 0.06	

TABLE VIII
QUALITATIVE SECURITY COMPARISON

Threat	Classical ($N = 5$)	Naive PQC	Amortized PQC ($N = 5$)
Harvest-now-decrypt-later	Not protected (X25519 secrets recoverable by a future quantum adversary) (no test: property of the primitive)	Protected (ML-KEM-768) `wire_sizes`	Protected (ML-KEM-768) `wire_sizes`

Active MITM on the handshake	Protected[†] (Ed25519, both directions)
	`forged_server_response_rejected`
	`substituted_kem_key_rejected`
		Protected (ML-DSA-65, both directions)
		`forged_server_response_rejected`
		`substituted_kem_key_rejected`
			Protected (ML-DSA-65, both directions)
			`forged_server_response_rejected`
			`substituted_kem_key_rejected`

Replayed update	Protected `replayed_update_from_old_round_rejected`	Protected `replayed_update_from_old_round_rejected`	Protected `replayed_update_from_old_round_rejected`

Reordered / duplicate update	Protected
	`reordered_update_rejected`
	`rollback_to_earlier_round_rejected`
		Protected (future-round key does not exist yet)
		`replayed_update_from_old_round_rejected`
			Protected
			`reordered_update_rejected`

Tampered ciphertext	Protected `tampered_ciphertext_rejected`	Protected `tampered_ciphertext_rejected`	Protected `tampered_ciphertext_rejected`
Session-key compromise exposure window	Bounded by N rounds (whole session if keyed once) `old_epoch_key_rejected_after_rekey`	Protected beyond 1 round (fresh key every round) `roundtrip_and_rekey_schedule`	Bounded by N rounds `old_epoch_key_rejected_after_rekey`

Test names refer to `tests/test_crypto.py` (prefix `test_` omitted). [†]Against classical adversaries; a quantum-capable active attacker could forge Ed25519 in real time. Exposure window: updates readable after compromise of one session key K_e; long-term identity-key compromise exposes no past updates in any mode (fresh KEM keys per epoch) but allows impersonation in later handshakes.

TABLE IX
AUTOMATED TESTS (PYTEST); PARAMETRIZED CASES ARE COUNTED PER TEST

Test	Purpose	Result
Crypto layer (`tests/test_crypto.py`)		
`roundtrip_and_rekey_schedule`	Round trip over 7 rounds; handshake count follows the mode's re-key schedule	pass (4/4)
`nonces_unique_and_ciphertexts_differ`	50 encryptions of one plaintext use 50 distinct 96-bit nonces	pass (4/4)
`partial_participation_late_joiner`	Client joining late keys the epoch the schedule prescribes	pass (2/2)
`encrypt_without_session_raises`	Encryption is refused before a handshake	pass (4/4)
`tampered_ciphertext_rejected`	Bit flips in ciphertext, tag or nonce, truncation, client-ID swap fail the AEAD	pass (20/20)
`replayed_update_from_old_round_rejected`	Old-round update rejected verbatim, with rewritten header, and as a duplicate	pass (4/4)
`reordered_update_rejected`	Update for a future round is rejected	pass (3/3)
`rollback_to_earlier_round_rejected`	Fresh update for a round before the last accepted one is rejected	pass (1/1)
`old_epoch_key_rejected_after_rekey`	Update under the pre-re-key session key is rejected	pass (2/2)
`update_with_wrong_epoch_header_rejected`	Update whose header epoch disagrees with the schedule is rejected	pass (2/2)
`server_rejects_update_when_client_skipped_rekey`	Client that skipped a scheduled re-key is rejected	pass (2/2)
`handshake_for_wrong_epoch_rejected`	Handshake signed for a different epoch is rejected	pass (2/2)
`replayed_handshake_rejected`	Replayed old handshake cannot overwrite a live session	pass (2/2)
`impostor_signing_with_own_key_rejected`	Handshake signed with a key not registered for the claimed ID is rejected	pass (4/4)
`bitflipped_signature_rejected`	Corrupted or empty client signatures are rejected	pass (4/4)
`substituted_kem_key_rejected`	MITM substitution of the client's KEM public key is rejected	pass (4/4)
`signature_does_not_transfer_across_epochs`	A signature from one epoch cannot authorize another	pass (2/2)
`unknown_client_rejected`	Handshake from an unregistered client ID is rejected	pass (4/4)
`forged_server_response_rejected`	Client rejects a MITM re-encapsulated or re-signed server response	pass (4/4)
`invalid_config_rejected`	Invalid mode / re-key interval combinations are rejected	pass (4/4)
`config_accepts_mode_strings`	Modes can be given as strings; naive PQC fixes $N = 1$	pass (1/1)
`classical_rekey_interval_accepted`	Classical mode accepts key-once or a re-key interval	pass (1/1)
`server_signs_response_in_every_mode`	Server response is signed (Ed25519 / ML-DSA-65) in every mode	pass (4/4)
`wire_sizes`	Key, ciphertext and signature sizes match FIPS 203/204; 16-byte GCM tag	pass (1/1)
FL harness (`tests/test_fl.py`)		
`flat_params_roundtrip`	Model parameters survive flatten / restore; size mismatch is rejected	pass (1/1)
`update_serialization_roundtrip_and_validation`	Update (de)serialization round trip; wrong size, truncation, NaN rejected	pass (1/1)
`fedavg_is_sample_weighted`	FedAvg computes the sample-weighted mean	pass (1/1)
`splits_partition_the_data`	IID and Dirichlet splits are disjoint and cover the dataset	pass (2/2)
`metered_suite_times_ops_and_interoperates`	Instrumented suite times each primitive and derives identical keys	pass (1/1)
`harness_rekey_schedule_and_csv`	Harness handshake schedule, CSV schema and per-component byte counts	pass (4/4)
`harness_is_deterministic_and_mode_independent`	Same seed gives identical accuracy in every mode	pass (1/1)
`harness_drops_tampered_and_replayed_updates`	Tampered and replayed updates are excluded from aggregation	pass (1/1)
`harness_keeps_global_model_when_all_updates_rejected`	If every update is rejected the global model is unchanged	pass (1/1)

98/98 test cases passed.